

CPE 4850 - Senior Project Design, Spring 26

Final Project Report

FlexType EMG Keyboard

Half-Byte Solutions

Date: May 8, 2026

Electrical and Computer Engineering

Kennesaw State University

Faculty: Dr. Jeffrey L Yiin

Table of Contents

Project Summary.....	1
Motivation.....	1
Proposed Solution.....	1
Key Outcomes	2
Impact Analysis	2
Product Description	2
Overview	2
Key Features.....	2
Design Considerations	3
Prototype Image	3
Hardware System Design	3
Overall System Block Diagram	4
Electrode Board Subsystem	4
Control Board Subsystem	5
Raspberry Pi Subsystem.....	5
PCB Schematic Design.....	5
Final Bill of Materials.....	7
Encasing Design.....	9
Software System Design	10
Overview of the Software Systems.....	10
Software Flow Chart	14
Key Algorithm Development.....	17
Signal Processing.....	18
1D-CNN Classification	18
EMA Voting Algorithm	18
Word Prediction and State Management.....	18
Verification.....	18
Device Driver Software	19
MindRove EMG Sensor Interface.....	19
Data Acquisition and Buffer Management	19
Real-Time Signal Conditioning Pipeline	19

Final Project Report

Communication Module Interface (REST API)	19
Resource and Session Management	20
AL/ ML System	20
Dataset Description.....	20
Model Architecture	21
Training Process	22
Evaluation Metrics	22
User Application Design	24
Overview of the User Interface (UI).....	24
Primary Functions and Controls.....	25
Technical Implementation Details	25
Results, Performance, and Requirements Compliance	26
Results Summary and Evidence	26
Product Requirement Compliance.....	26
Improvements and Optimizations	29
Conclusions and Lesson Learned	29
Appendix	30

Table of Tables

Table 1 Control Board Bill of Materials.....	7
Table 2 Electrode Board Bill of Materials (x5)	8
Table 3 Gesture Commands and Prediction Accuracies	11
Table 4 Gesture Coordinate Map.....	14
Table 5 Gesture Thresholds for EMA Voting.....	16
Table 6 Gesture Prediction Model Architecture	21

Table of Figures

Figure 1 Device Prototype Worn on Arm.....	3
Figure 2 Overall Hardware System Block Diagram	4
Figure 3 Electrode Board PCB Design	6
Figure 4 Control Board PCB Design.....	7
Figure 5 Control Board 3D Encasing.....	10
Figure 6 Gesture Prediction Model Training Flowchart.....	14
Figure 7 Driver Flow Chart	15
Figure 8 EMA Voting Flowchart	17
Figure 9 EMG Average Time Domain Signals	21
Figure 10 Gesture Prediction Testing Accuracy	23
Figure 11 Gesture Prediction Model Loss	23
Figure 12 Gesture Prediction Confusion Matrix	24
Figure 13 UI Terminal Output During Typing.....	25
Figure 14 UI Terminal Output During Word Prediction and Selection	25
Figure 15 Knock Gesture Output	27
Figure 16 Output of Thumb Out Gesture Deleting Most Recent Letter Group	27
Figure 17 Output of L-Sign Swiping to the Next Word	28
Figure 18 Gesture Prediction Output with Time Information	28

Project Summary

The FlexType EMG Keyboard is a wearable wristband device that can detect the electromyography (EMG) signal from the muscles in your arms and convert them into text inputs. This device is designed for individuals with neurological disorders that limit hand or arm mobility and provides an accessible, versatile, and private alternative to traditional keyboards. By using a 1D convolutional neural network (1D-CNN) model trained to recognize different hand gestures, each corresponding to a different set of letters and commands, we can give independence back to disabled individuals who often rely on others to type.

Motivation

Traditional keyboards acquire a certain level of fine motor control that a large portion of the population can't reliably perform. According to the World Health Organization, a third of the human population are affected by neurological conditions, making it the leading cause of disabilities worldwide. According to the U.S. Census Bureau, nine million people in the United States have difficulty with skills like lifting and grasping, which directly impacts their ability to type. Unfortunately, these individuals find traditional keyboards frustrating and often completely unusable. If they try using a keyboard, it takes them significantly more time to communicate their messages, having to redo what they type countless times over due to misclicks or typos. If they seek traditional alternatives like voice-to-text, then they sacrifice privacy and may only be able to communicate in quiet environments. Most voice-to-text solutions also do not work well with people who struggle with diseases like Parkinson's, which affect the vocal chords. Eye-tracking keyboards are also often very expensive, costing over \$1,000, and are difficult to carry around for the average user. Advanced solutions like the Meta Ray-Ban Display smart glasses are also often expensive, costing around \$500, making them unaffordable for the average person. These solutions are also not specialized to typing and may lack in accuracy or ease of use.

Proposed Solution

Our solution is the FlexType EMG keyboard, a wearable wristband of 5 custom EMG electrode boards wrapped around the forearm, connected to a control board that digitizes and sends the data to a Raspberry Pi running two machine learning models. Users will perform defined gestures, each corresponding to a group of letters or specific commands, to type out their desired words and phrases with ease. Our gesture machine learning model classifies the incoming gestures performed and uses the word prediction model to predict the most likely intended word.

Unlike voice to text, the FlexType EMG keyboard requires no vocalization or fine motor control, which is extremely useful for individuals with disabilities that impact their dexterity and voice. Unlike eye-tracking devices that can be very expensive and bulky, the FlexType EMG keyboard is designed to be a small wearable device that is cheap, easy to use, and easy to wear and carry. The FlexType EMG keyboard gives people a reliable, private, and low effort alternative to the traditional keyboards and inaccessible typing assistants often used by our target market.

Key Outcomes

Our proposed solution was able to achieve the goal of accurately typing words and phrases based on gesture commands. Our gesture prediction model performed with ~93% accuracy, which surpassed our goal of 90% accuracy. Our implemented driver was also able to successfully add, delete, enter, and swipe between predicted words using the mapped gesture commands. We proved that our proposed solution can make typing more accessible to people who would otherwise struggle with standard keyboards and voice-to-text typing.

Impact Analysis

The FlexType EMG keyboard offers a positive, much needed impact to people who struggle with neurological conditions. Traditional accessibility solutions are often difficult for the people who need them most, and many people rely on the help of caregivers or loved ones to type for them.

Our product bridges the gap in accessibility, affordability, and privacy by providing a lightweight, low-effort wearable device that can restore independence to those who have lost it. This device is also accessible to a wide margin of our target audience because it is easy to carry, is not dependent on environmental factors like noise, and is significantly cheaper than other alternatives, costing only \$161.24. Our device could have a significant, positive affect on people with disabilities because restored independence will mean improved mental health and overall quality of life by putting the power of communication back in their hands.

Product Description

Overview

The final product consists of a wearable EMG sensor that collects EMG data from the user's arm. It then sends this data to the Raspberry Pi 4, where it uses the gesture prediction model to predict the gesture the user is performing. The gestures correspond to different letter groups and commands. After the user performs all the desired letter group gestures and the "Enter" gesture, the letter groups get sent to the remote word prediction model where the three most likely words are generated. The user can perform the "Swipe" command to select the desired word from the predicted words list. Lastly, the desired word and full sentence are displayed on the RPi terminal.

Key Features

- The product collects EMG data from the user's arm
- The product uses a gesture prediction model to recognize eight different gestures and a "Rest" gesture
- The product converts recognized gestures into coordinates for letters and commands
- The product can access a remote server that contains the word prediction model
- The product predicts the three most likely words the user is trying to type
- The product displays the three most likely words, the chosen word and the full sentence typed
- The product allows the user to select a word from the three options
- The product allows the user to delete a word or letter group during typing

Design Considerations

- **Space Efficiency:** The FlexType EMG Keyboard was designed to be worn on the forearm, requiring the PCBs and the 3D enclosures to be as compact as possible.
- **Signal Integrity:** Using the INA333 instrumentation amplifier and OPA2320 op-amp was to ensure clean signal acquisition before digitization. The INA333 removed noise and cleaned the EMG signal while the op-amp filtered the signal to only useful data removing high frequencies.
- **Dual PCBs:** Designing an electrode board and control board allowed for the hardware to be distributed across the forearm instead of having a single bulky device. We wanted multiple EMG channels so if the boards weren't separated, it would have been impossible to fit enough electrodes on the arm.
- **Comfort and daily wearability:** Using stainless steel binding screws as electrodes allows us to not need conductive gel. Allowing our device to be easily used every day without worrying about any preparations.
- **Limited Volunteers for Data Collection:** We were unable to gather many individuals to collect EMG data for our gesture prediction model, so we had to resort to collecting a large number of iterations from one specific subject and tailored the model to them.

Prototype Image



Figure 1 Device Prototype Worn on Arm

Hardware System Design

The FlexType EMG Keyboard has three primary subsystems working together: our electrode board which gathers to analog EMG data from the user, the control board that digitizes the data to be read by our microcontrollers, and the Raspberry Pi responsible for operating the gesture and word prediction models.

Overall System Block Diagram

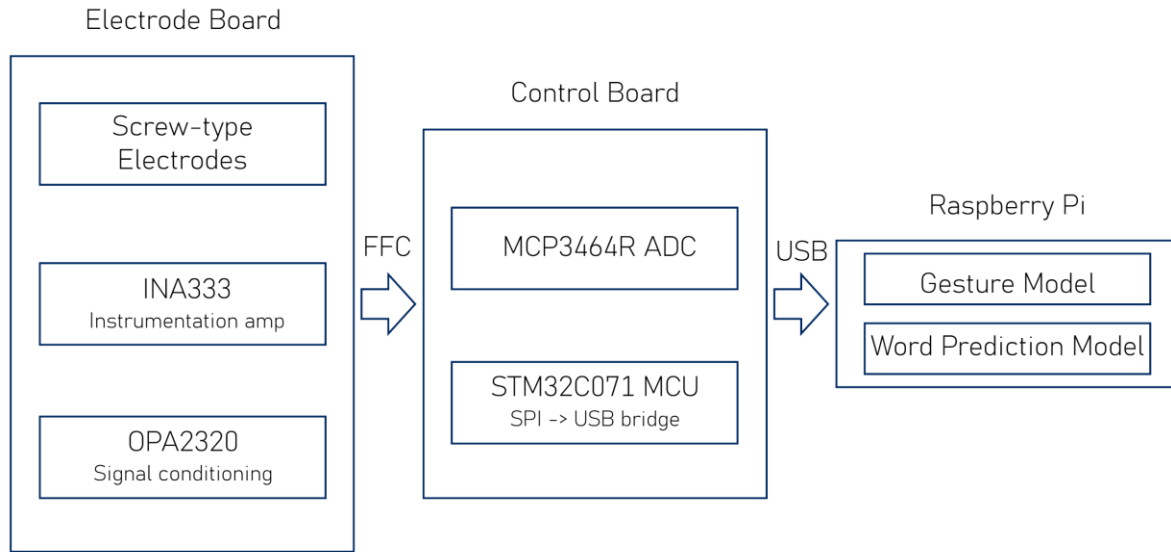


Figure 2 Overall Hardware System Block Diagram

Electrode Board Subsystem

The electrode board is the analog front end of the FlexType. Its function is to acquire raw EMG signals from the forearm via dry strew type electrodes and condition them into a clean amplified signal for ADC conversion.

The electrodes make direct contact with the forearm to gather the signals. The INA333 receives the signals for amplification, operating in differential, taking the signal from both electrodes and removing noise such as 50/60Hz. The signal is then amplified by nearly 1000x with the use of our 100-ohm resistor. This is done because the INA333 has two 50k ohm resistors internally, so connecting the 100-ohm resistor forms a relationship between the resistors resulting in a 1000x gain.

The OPA2320 op-amp comes next, which shapes the frequency response of the EMG signal so it can be used for ADC conversion. The op-amp also serves as our anti-aliasing filter to remove any high frequency noises.

The main challenges had to do with being able to gather clean useful signals from the strew type electrodes. Dry electrodes give off noisier signals than wet electrodes, so we had to select components that will help combat that noise and clean it up. The INA333 and OPA2320 were picked just for that, the INA333 because of its high input impedance and the OPA2320 for further ensuring any noise introduced at any point of the signal transfer was removed.

Control Board Subsystem

The control board is our central hub, deals with digitizing the conditioned EMG signals from the electrode boards and sending the resulting data to the Raspberry Pi over USB. The board contains the ADC, the microcontroller, and the power regulator.

Our ADC the MCP3464R receives the conditioned signals from the electrode board over the flat flexible connector (FFC) cables. It communicates with our STM32 microcontroller over SPI. The microcontroller acts as the SPI to USB bridge. The data that is sent from the ADC is repackaged as USB data, so it can be sent to the Raspberry Pi via the USB-C port. The data is read on the raspberry pi using standard serial libraries.

As for power, that is handled by the MCP1754ST-3302E/CB low dropout voltage regulator, which gives the entire system a clean 3.3V power source to work with.

The biggest challenge was fitting all the necessary components within the smallest possible footprint. As a wearable device, a large board meant a larger enclosure which could affect comfortability for the user wearing it.

Raspberry Pi Subsystem

The Raspberry Pi 4 primarily serves as the processor for the gesture signals and acts as the user's device. The Raspberry Pi contains the gesture prediction model in the form of a .keras file, and runs the driver of the device, which processes the EMG signals through the gesture prediction model and converts the signals to different commands.

The biggest challenge was connecting our designed PCBs to the Raspberry Pi. Due to delays in receiving components and building our hardware, we modified this subsystem to work with a commercial MindRove EMG armband. The MindRove connects to the Raspberry Pi via a USB WiFi dongle and uses eight channels instead of five. The commercial MindRove was also used to capture sample EMG data from volunteers, so it worked well with our designed gesture prediction model.

PCB Schematic Design

The electrode board PCB design is seen in Figure 3. The electrode board is responsible for gathering and collecting the raw EMG signals from the user's forearm. It was designed with 0603 SMD components to minimize its size as small as possible since our device is wearable. The screw-type electrodes are mounted directly through the PCB, which was selected so we didn't have to use conductive gel and makes the device more practical for repeated daily use.

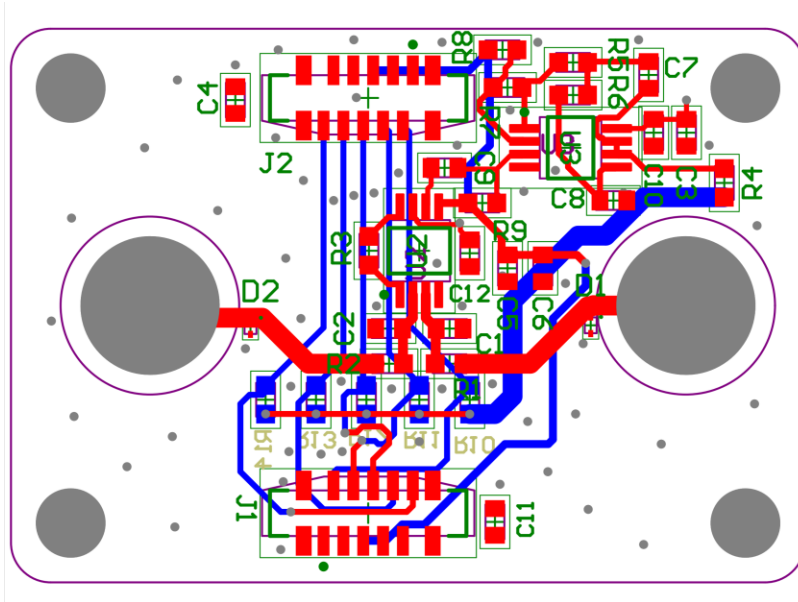


Figure 3 Electrode Board PCB Design

The signal from the electrodes first encounters the INA333 instrumentation amplifier. Which measures the difference between the two electrode inputs and removes any noises common to both. The signal is then amplified by 1000x, so it can be useful for data. That data is then passed to the OPA2320 op-amp, which amplifies and filters the signal even more. The op-amp filters the signal to be only the EMG frequency we need, which is a range of 20Hz to 500Hz. Our final signal is passed to the control board using FFC connectors and cables.

The PCB design for the control board is seen in Figure 4. The control board is responsible for receiving the filtered/clean analog EMG signals from our electrode boards, digitize them, and stream the end data to the Raspberry Pi to be used in our machine learning models.

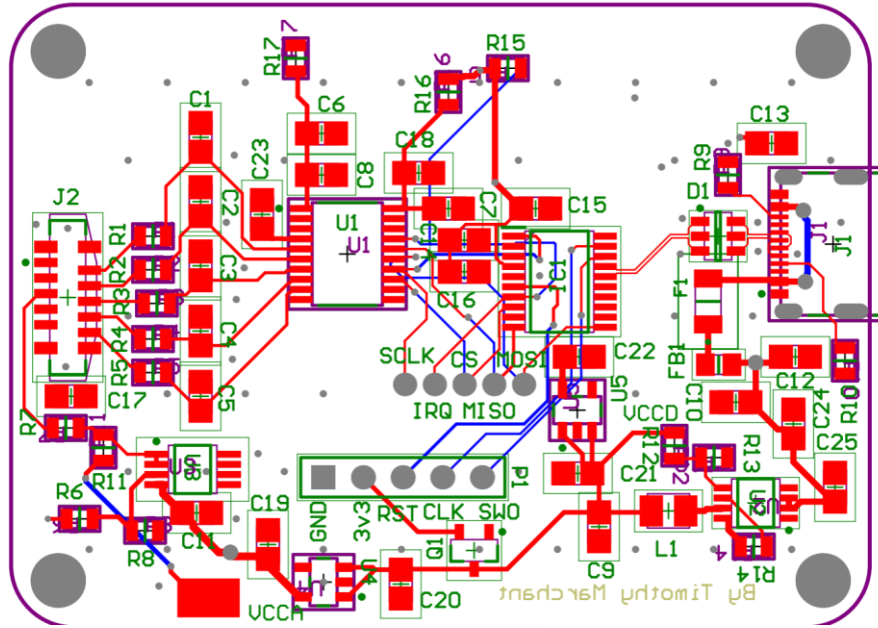


Figure 4 Control Board PCB Design

The analog signals from the electrode boards are fed to the MCP3464R ADC to be digitized and sent to our microcontroller over an SPI interface. The STM32 microcontroller acts as the SPI to USB bridge between the ADC and our Raspberry Pi. It reads the data from the ADC over SPI and repackages it as USB, so the data can be sent to the Raspberry Pi via the control board's USB-C port.

The power for the whole system comes from the control board where the on-board voltage regulator gives the system a stable 3.3V. This is not only for the control board but also powers the electrode boards through the FFC cables.

Final Bill of Materials

Table 1 Control Board Bill of Materials

Component	Qty	Price
ADC MCP3464R-E/ST	1	\$4.91
STM32C071FBP6	1	\$1.40
USB-C Port	1	\$0.83
Voltage Regulator	1	\$1.98
FFC Connector	1	\$1.67
65k ohm resistors	5	\$0.05

100k ohm resistors	3	\$0.03
10 ohm resistors	3	\$0.03
5.1k ohm resistors	2	\$0.28
1M ohm resistors	1	\$0.15
820k ohm resistors	1	\$0.17
200k ohm resistors	1	\$0.10
10k ohm resistors	1	\$0.10
4.7nF capacitor	5	\$0.25
0.1uF capacitor	9	\$0.63
10uF capacitor	4	\$0.56
1uF capacitor	4	\$0.52
4.7uF capacitor	1	\$0.34
22uF capacitor	1	\$0.19
47uF capacitor	1	\$0.61
Other components	8	\$2.7
Total		\$17.50

Table 2 Electrode Board Bill of Materials (x5)

Component	Qty	Price
INA333	5	\$22.70
OPA2320A	5	\$20.35
FFC Connectors	10	\$14.20
47k ohm resistors	10	\$0.10
10k ohm resistors	15	\$0.15
100 ohm resistors	5	\$0.50

3.3k ohm resistors	10	\$0.10
100k ohm resistors	5	\$0.50
0 ohm	25	\$0.25
47pF capacitor	10	\$0.80
0.1uF capacitor	35	\$1.40
4.7uF capacitor	10	\$2.60
1uF capacitor	5	\$1.15
Screw type electrodes	1 set	\$10.99
Raspberry Pi 4	1	\$67.95
Total		\$143.74

Encasing Design

The encasing was designed as a compact and protective enclosure for the electronic PCB components, providing the structural support and cable management needed within a 40mm x 28mm footprint. The enclosure consists of two different custom 3D printed parts: a main body and a removable lid. Both are secured together via M2 screws, as the PCB is secured to the enclosure via M3 screws.

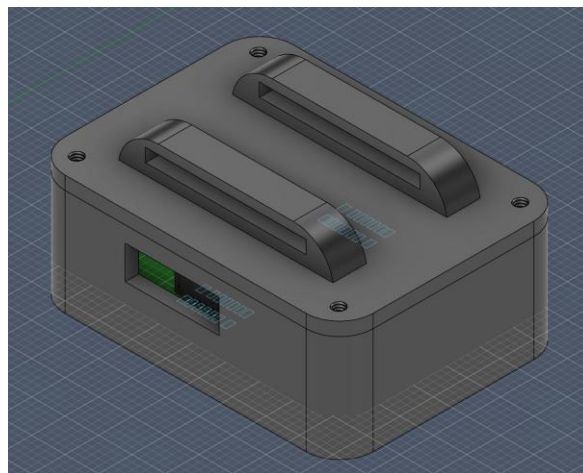


Figure 4 Electrode Board 3D Encasing

As seen in Figure 4, the enclosure houses the internal electronics while allowing cables to enter and exit the enclosure. A USB-C port cutout for power transfer was included for the control board enclosure as well as opening for FFC cables and dedicated electrode mounting points on the enclosure to ensure

proper contact between the electrode and the user. These included features ensure accessibility while maintaining enclosure support structurally and component protection.

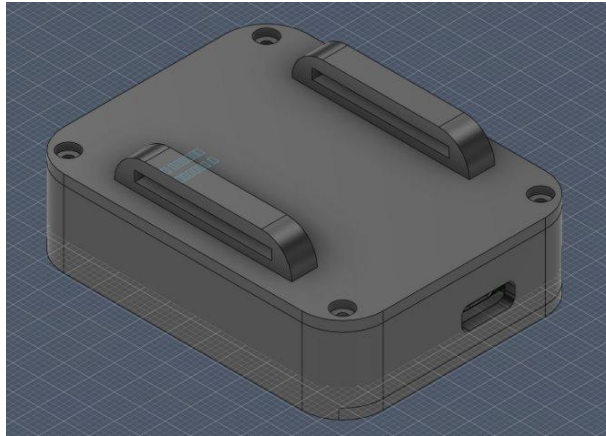


Figure 5 Control Board 3D Encasing

The enclosure for the control board didn't need to be as compact as the electrode board since it would be separated from the rest of the boards. The biggest issue with designing the control board had to be the USB-C cut out. If designed to be the same size as the port, a USB-C cable would not reach the port of the board to be able to connect. Due to this, most of the control board iterations came from slowly increasing the size of the USB-C cut out to achieve proper fitment.

Software System Design

The software system of the FlexType EMG Keyboard primarily consists of our custom designed and trained gesture prediction model and the driver that runs the full connected device.

Overview of the Software Systems

The driver for the FlexType EMG Keyboard is separated into four main sections. First is EMG data processing, then passing the EMG data to the gesture prediction model, then converting the gestures to the various commands, and finally passing the full string of letter groups to the word prediction model. The diagram of this subsystem can be seen in Figure 5.

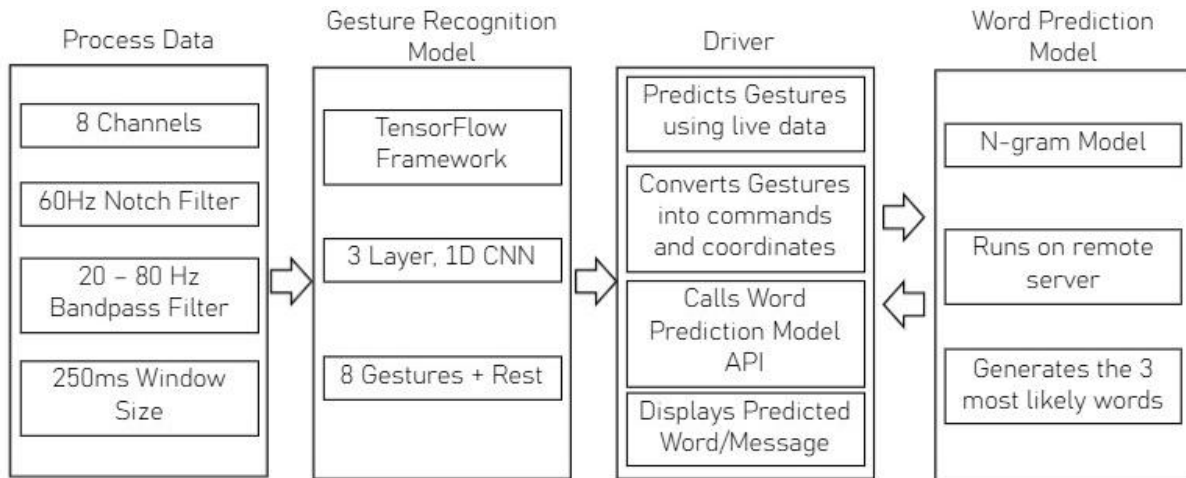


Figure 5 Driver Software Diagram

The user can wake the device with a “Surfs Up” gesture command to begin typing. Once the device starts running, the first step is to take in the raw EMG data from the wearable EMG armband, which consists of 8 channels. We then process the data with a 60Hz notch filter, a 20-80Hz bandpass filter, and a window size of 250ms. This converts the raw data into data that we can interpret and categorize into distinct gesture labels.

When the data has finished processing, we pass it into the gesture prediction model. The gesture prediction model uses a TensorFlow Framework is a 1D-CNN model with 3 layers. The model is trained to classify eight unique gestures and a “Rest” gesture. An exponential moving average (EMA) voting algorithm is used to balance and boost prediction accuracy by enhancing “weak” gestures and limiting “strong” ones. Table 3 shows each gesture, its associated command, and its prediction accuracy.

Table 3 Gesture Commands and Prediction Accuracies

Gesture Name	Gesture Image	Command	Prediction Accuracy
Rest		-	98.65%

Middle Pinch	 A hand with the thumb and index finger pinched together, and the other three fingers extended.	A-F	81.74%
Ring Pinch	 A hand with the thumb and ring finger pinched together, and the other three fingers extended.	G-M	96.94%
Pinky Pinch	 A hand with the thumb and pinky finger pinched together, and the other three fingers extended.	M-T	85.26%
Three Fingers	 A hand with the index, middle, and ring fingers extended, and the thumb and pinky finger pinched together.	U-Z	84.21%

L-Sign		Swipe	99.14%
Thumb-Out		Delete	92.88%
Knock		Space/ Enter	98.69%
Surfs Up		Start/ Stop	86.90%
		Overall Accuracy:	93.08%

Once the gesture prediction has been made, it is converted to a set of coordinates for the FlexType word prediction model. The current series of predicted letter groups are also displayed to the user so that they can preview what letters are being passed in. If an incorrect gesture is predicted or if the user makes a mistake, they can delete the previous prediction by doing the “Thumb-Out” command.

Once the user submits the word for prediction, the set of letters are passed in for word prediction with the FlexType model. The reason the letter groups are passed in as coordinates is because the FlexType model was initially designed to work with a touch screen and different “taps”. Table 4 shows how the different gestures are mapped to coordinates before being passed to the FlexType model.

Table 4 Gesture Coordinate Map

Gesture Name	Letter Groups	Coordinates
Middle Pinch	A-F	[0, 0]
Ring Pinch	G-M	[100, 0]
Pinky Pinch	N-T	[200, 0]
Three Fingers	U-Z	[300, 0]
Knock	Space	[-100, -100]

After the FlexType API is called, the top three predicted words are returned to the driver. The user can then use the “L-Sign” gesture to swipe to the word they wanted to type. The word prediction model is ~95% accurate and users ten characters of context when making predictions. The user will then “Knock” to confirm the word prediction and can continue on to the next word. The process will continue until the user has finished typing. When the user is done typing, they can do the “Surfs Up” command to end the program.

Software Flow Chart

Figure 6 shows the flowchart for building, training, and testing our gesture prediction model. More details about the gesture prediction model can be found under the AL/ ML System section.

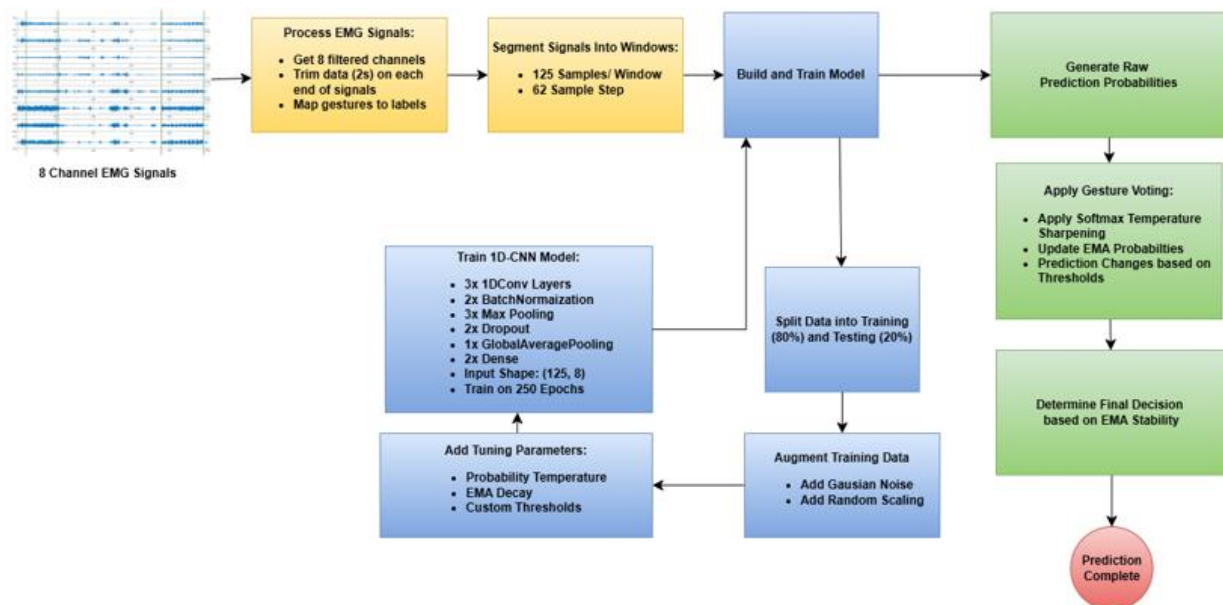


Figure 6 Gesture Prediction Model Training Flowchart

Figure 7 shows the flowchart of the main driver of our program. The driver switches between the various states based on user's gesture commands. The user can start or end the program by performing the "Surfs Up" gesture.

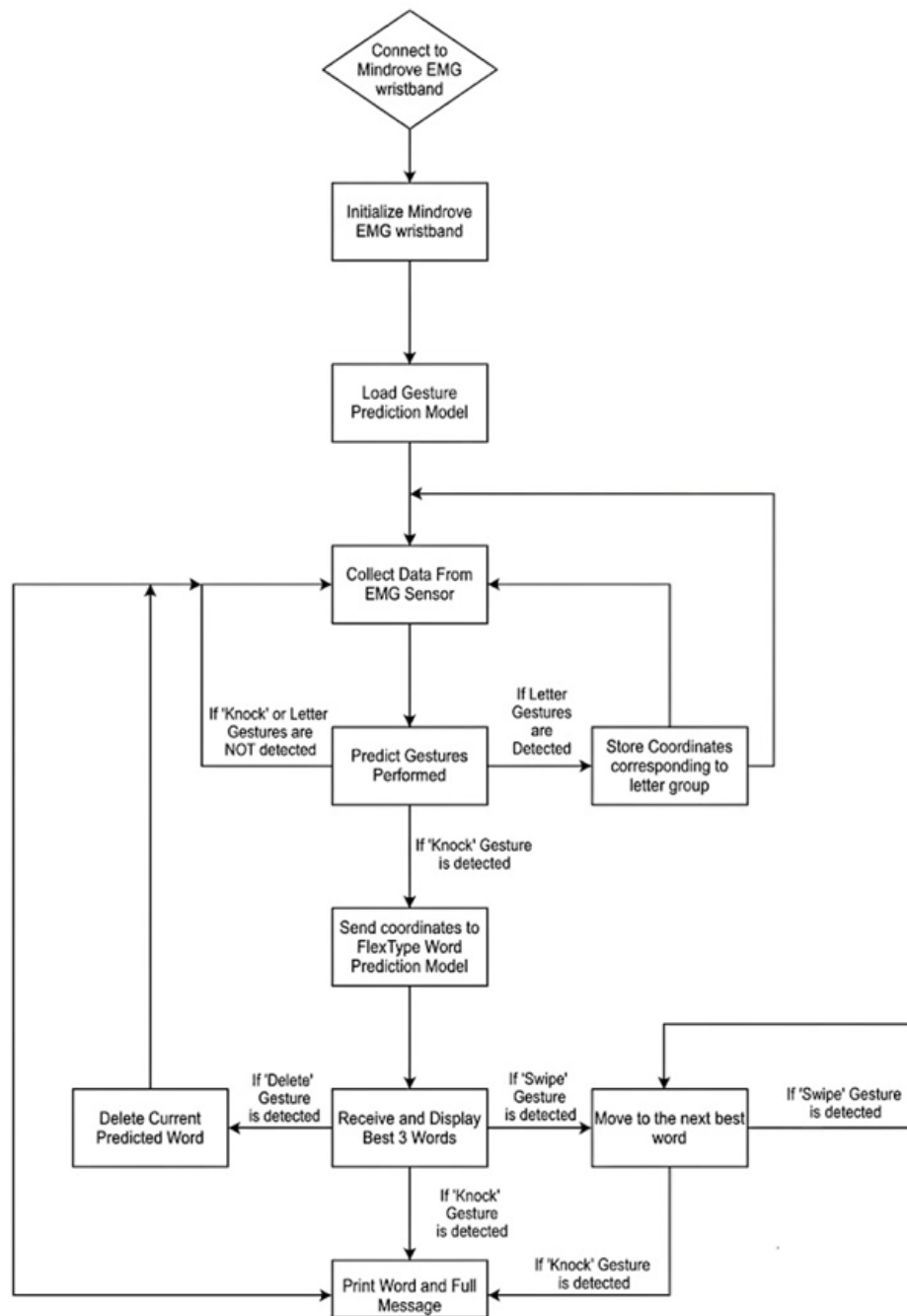


Figure 7 Driver Flow Chart

Figure 8 ~~Figure 5~~ shows the flowchart for the EMA voting algorithm. Each gesture must be predicted with a confidence greater than its assigned threshold to be voted as the best gesture. Because some gestures are more easily predicted than others, custom thresholds are assigned to make predicting that

gesture easier or harder accordingly. The tuned gesture thresholds are seen in Table 5 Gesture Thresholds for EMA Voting

Table 5 Gesture Thresholds for EMA Voting

Gesture	Threshold Value
Rest	0.45
Middle Pinch	0.05
Ring Pinch	0.40
Pinky Pinch	0.30
Three Fingers	0.15
L-Sign	0.70
Thumb-Out	0.40
Knock	0.75
Surfs Up	0.70

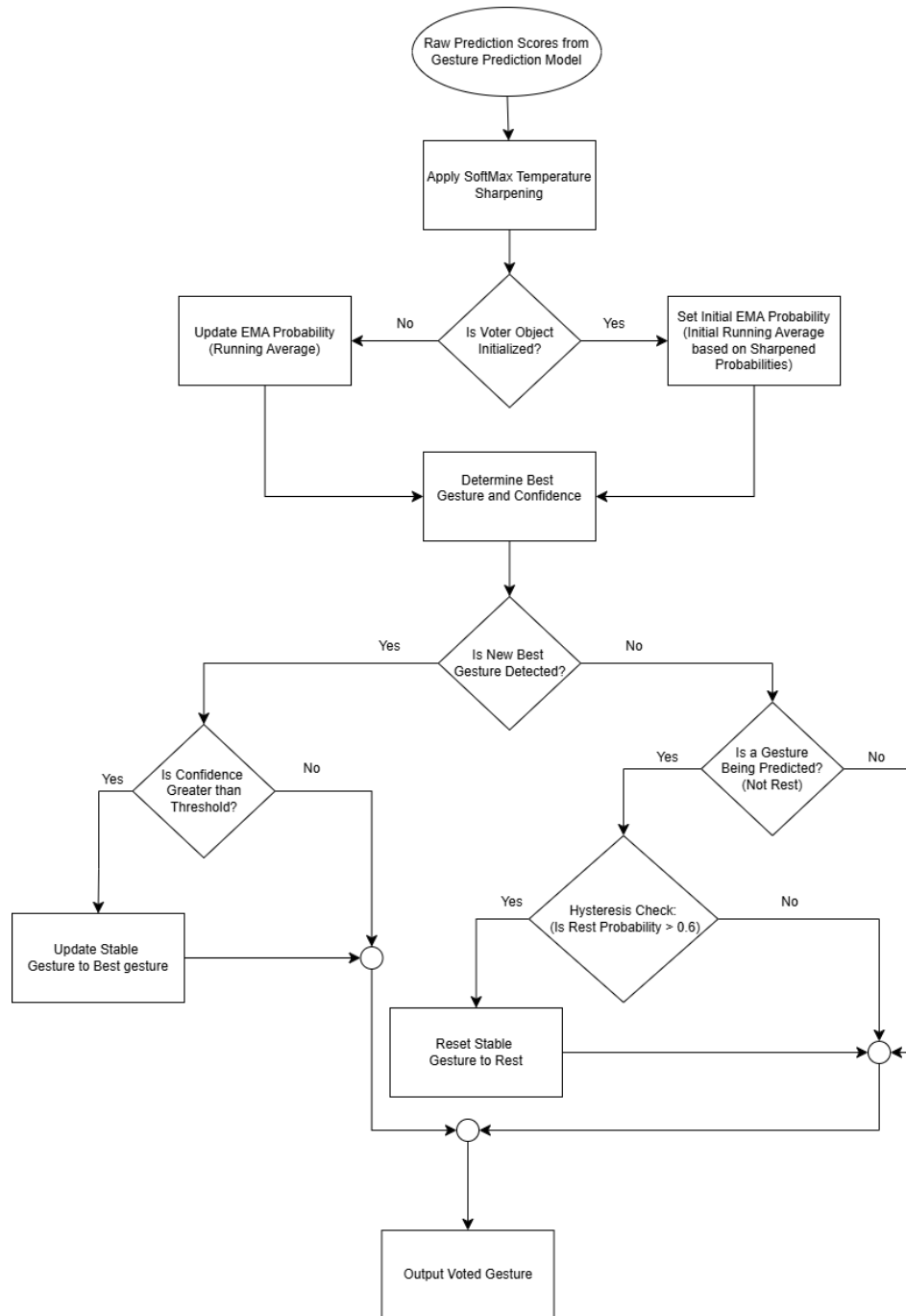


Figure 8 EMA Voting Flowchart

Key Algorithm Development

The architecture for the FlexType EMG keyboard utilizes a sequential pipeline that processes raw EMG data from eight channels, classifies them using deep learning, and translates the gesture to letter groups, then to words using the FlexType model.

Signal Processing

Raw EMG signals are highly affected by noise, motion, and placement. To isolate relevant frequencies from irrelevant or noisy ones, digital filtering is applied after data gathering. A 4th-order bandpass filter is applied between 20Hz and 80Hz to capture the core frequencies of the muscle movements. Readings outside the range are discarded and ignored. An infinite impulse response (IIR) notch filter is applied at 60Hz as well to eliminate electrical interferences.

1D-CNN Classification

A 1D-CNN model is used to translate the right filtered channels of EMG data. This model processes sliding windows of 125 samples and utilizes three layers of decreasing kernel sizes to learn both broad and specialized patterns over each signal. Regularization is also applied via Batch Normalization to prevent overfitting after each convolution, and dropout layers are implemented with a rate of 0.4. Lastly, GlassAveragePooling1D is used to reduce dimensionality before passing features to a fully connected dense layer with SoftMax activation. This outputs a raw probability across the nine classifications.

EMA Voting Algorithm

An EMA voting algorithm is implemented to act as a stabilizing filter over raw probabilities from the gesture prediction model. This helps reduce misclassifications and boost overall prediction accuracy. First, raw SoftMax probabilities are sharpened using a temperature value of 0.8 to exaggerate differences between the most likely gesture and second most likely gesture to reduce ambiguity. An EMA with a decay factor of 0.25 is then applied to sharpened probabilities to ensure that a single and frame of data does not greatly alter the current predicted state. Lastly, the currently recognized gesture is only updated if the smoothed confidence exceeds the gesture-specific threshold (seen in Table 5). Transitioning back to the “Rest” prediction requires the confidence to exceed a threshold of 0.6.

Word Prediction and State Management

A final state machine (FSM) is synchronized with the FlexType word prediction API to switch between gesture prediction and word prediction states. Specific gestures are mapped via assigned coordinates on a virtual keyboard (seen in Table 4) and appended to an array. The FSM controls which “mode” the system is in. The modes are “Typing”, “Checking”, and “Wait”. The “Surfs Up” gesture acts as a toggle to enable and disable typing. A “Rest” must be sensed between each gesture prediction to toggle between the “Wait” state on and off. This ensures that an accidental or misread gesture is not added while transitioning from one gesture to the next. When the “Knock” gesture is performed, the array of gesture coordinates, or “taps”, is passed as a JSON payload to the FlexType API and the “Checking” state is entered. Here, the user can select their predicted word with the “Swipe” command, delete the prediction, and re-enter the typing state by confirming their selection with a second “Knock”.

Verification

We validated the performance of the gesture prediction model by passing unseen data to the model and evaluating its accuracy based on the model’s predictions. Table 3 shows each gesture’s accuracy after testing and training.

We also evaluated the model based on its ability to correctly type words and phrases. Based on previous knowledge of the FlexType word prediction model, we already knew it had a ~95% accuracy. We validated this by typing the following words and phrases into the model:

- “hello”
- “it was fun”
- “i like birds”

The combined models in the driver were able to successfully type our test cases, indicating that it was functioning properly. We also verified that performing the “L-Sign” gesture to “Swipe” between word predictions and the “Thumb-Out” gesture to delete word and gesture predictions was functional during our use-case testing. Lastly, we validated the function of the “Surfs Up” command by starting and ending each test case with it.

Device Driver Software

For this device, a custom Python hardware interfacing script was developed to manage the flow of data from the EMG sensors to the local processing pipeline and API calls. This software is the bridge between real-time data collection, signal processing, and external API requests.

MindRove EMG Sensor Interface

The primary hardware component of this device is the MindRove EMG Armband and WiFi Board. This streams eight channels of EMG data to our connected Raspberry Pi via a WiFi dongle. Our driver then utilizes the MindRove API to establish a wireless connection and configure the board parameters. The data logging session is prepared and active EMG channels and sampling rate are retrieved directly from the board’s configuration.

Data Acquisition and Buffer Management

To ensure that the gesture prediction model is fed consistently and correctly sized windows of data, a blocking data acquisition loop was implemented. Our *get_emg_data(num_points)* function polls the board’s internal buffer and halts until exactly 125 samples are available. When the threshold is met, the most recent frame of data for the active EMG signals is extracted.

Real-Time Signal Conditioning Pipeline

Before raw collected data is passed for classification, a *process_data()* function is called using the *scipy.signal* library. The previously described bandpass and IIR notch filters are applied and the data is now ready to be passed to the gesture prediction model.

Communication Module Interface (REST API)

To translate local gesture predictions to text predictions, a communication driver is implemented to interface with the remote FlexType API at *imagineville.org*. The *flextype()* function converts “taps” into a structured JSON payload. The *requests* library executes POST commands and incorporates a 5-second timeout safeguard to prevent hanging during network drops. It also include exception handling in the event of HTTP or JSON parsing errors.

Resource and Session Management

To prevent memory leaks and locked ports, a *try...except* block is wrapped around the driver. If a *KeyboardInterrupt* signal is detected, the program terminates. The *stop_stream()* and *release_session()* functions are called to sever WiFi connection to the MindRove board.

AL/ ML System

The gesture prediction model is built upon a deep learning pipeline designed to interpret high-frequency EMG signals. It transforms the raw EMG data into discrete commands through a 1D-CNN and temporal stabilization layer.

Dataset Description

Our dataset includes samples from nine unique individuals, as well as nine samples taken from the same subject at different days and times and with varying EMG armband placement. For each subject and sample set, each gesture was performed over a period of seven seconds, with two seconds of “Rest”, three seconds of gesture holding, and another 2 seconds of “Rest”. This was repeated ten times for each gesture for each subject. We also captured two minutes of each subject holding “Rest” to act as a control for the model.

Each sample consists of data from eight channels at a sampling rate of 500Hz. The raw signals are segmented into windows of 125 samples (representing 250ms temporal slice) with a step size of 62 samples to allow for sufficient overlap. To ensure only active muscle signatures are trained on, the two seconds of lead-in and trailing “Rest” data is trimmed from each sample. The average signal of each gesture mapped in the time domain can be seen in Figure 9.

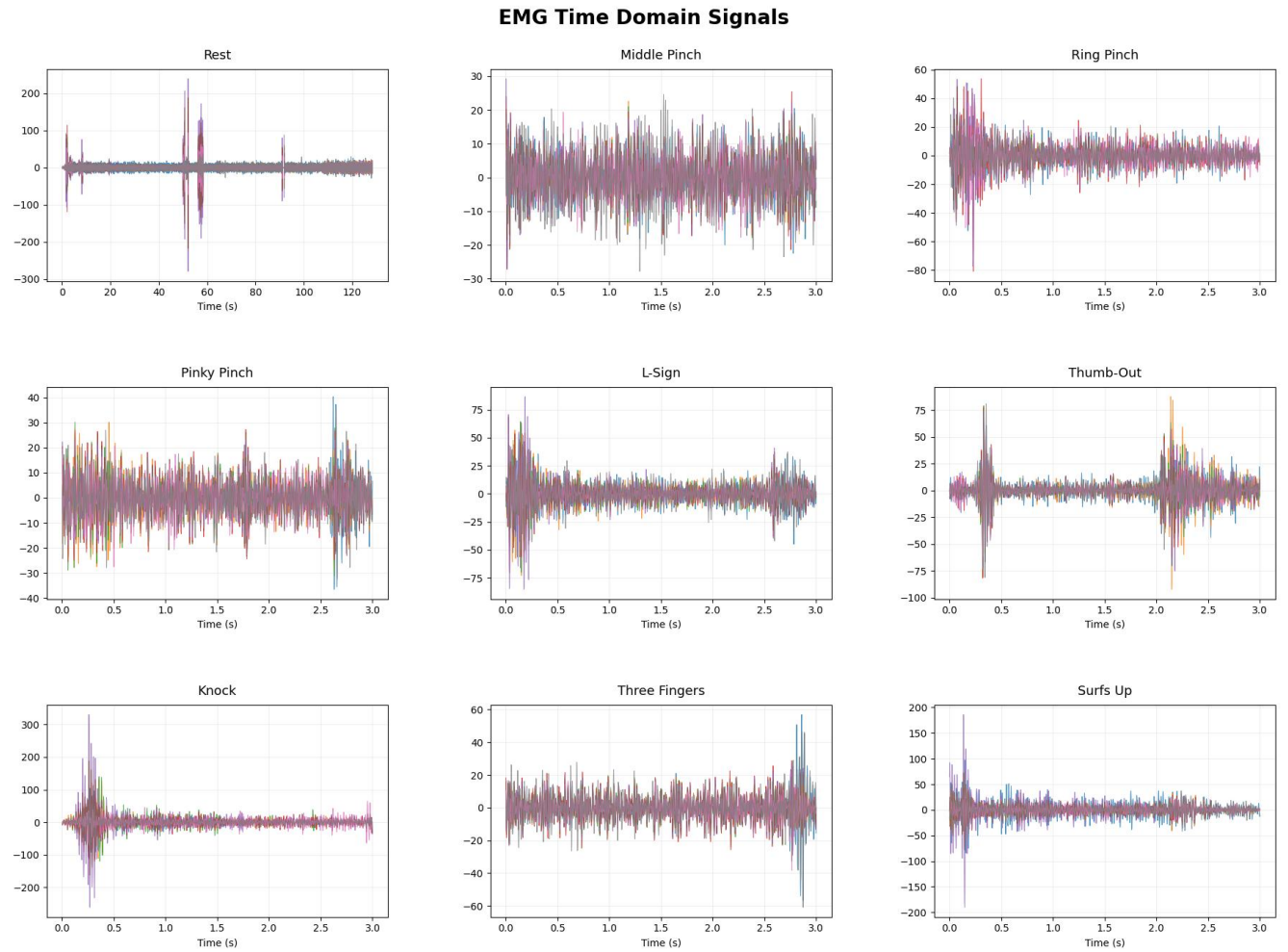


Figure 9 EMG Average Time Domain Signals

Model Architecture

Table 6 Gesture Prediction Model Architecture

Layer	Type	Configuration
Input	InputLayer	125 x 8 (Time Steps x Channels)
Feature Extraction	Conv1D	64 Filters Kernel Size 15 ReLU Activation
Downsampling	MaxPooling1D	Pool Size 2
Feature Extraction	Conv1D	64 Filters Kernel Size 9 ReLU Activation
Regularization	Dropout	Rate = 0.40

Feature Extraction	Conv1D	64 Filters Kernel Size 5 ReLU Activation
Pooling	GlobalAveragePooling1D	Flattens Temporal Dimension to Feature Vectors
Decision	Dense	64 Neurons ReLU Activation
Output	Dense	9 Neurons SoftMax Activation

Training Process

This model is training using an 80/20 train-validation split. To improve generalization and prevent the model from becoming overly sensitive to electrode placement or individual signal strength, a data augmentation strategy was implemented. Gaussian noise was added with a value of 0.01. Random scaling of signal amplitude with a value between 0.8 and 1.2 was also implemented. The optimization process utilizes the Adam optimizer with a Categorical Crossentropy loss function. Training was conducted over 250 epochs with a batch size of 32.

Evaluation Metrics

The performance of the gesture prediction model was performed through a combination of standard classification metrics and real-time stability analysis. The model accuracy calculated with the 20% split from the training dataset can be seen in Figure 10. The model loss during training can be seen in Figure 11.

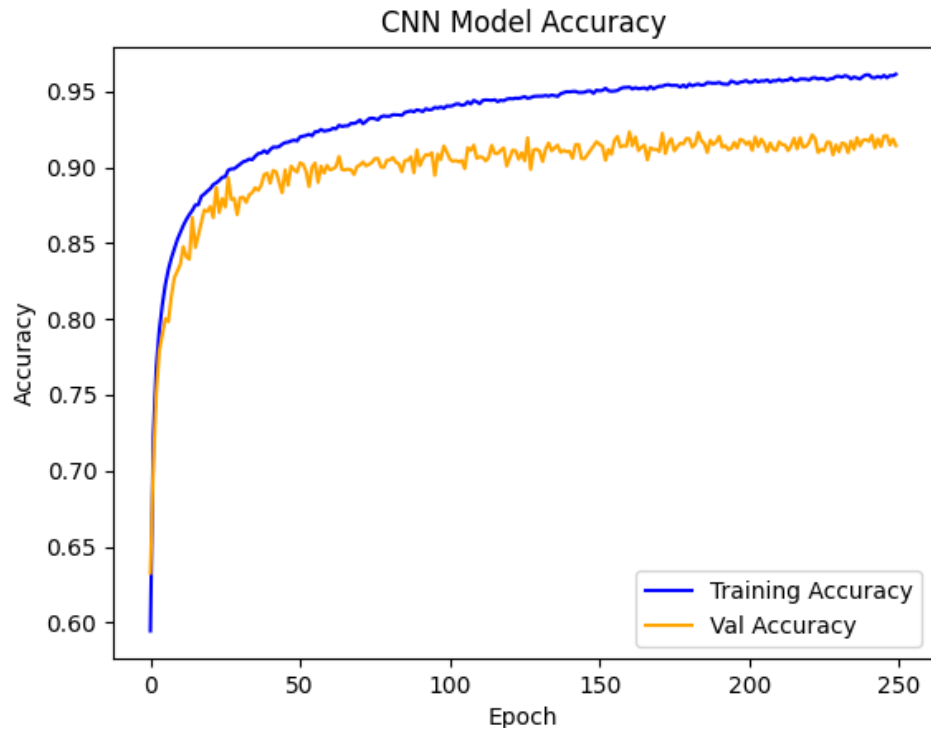


Figure 10 Gesture Prediction Testing Accuracy

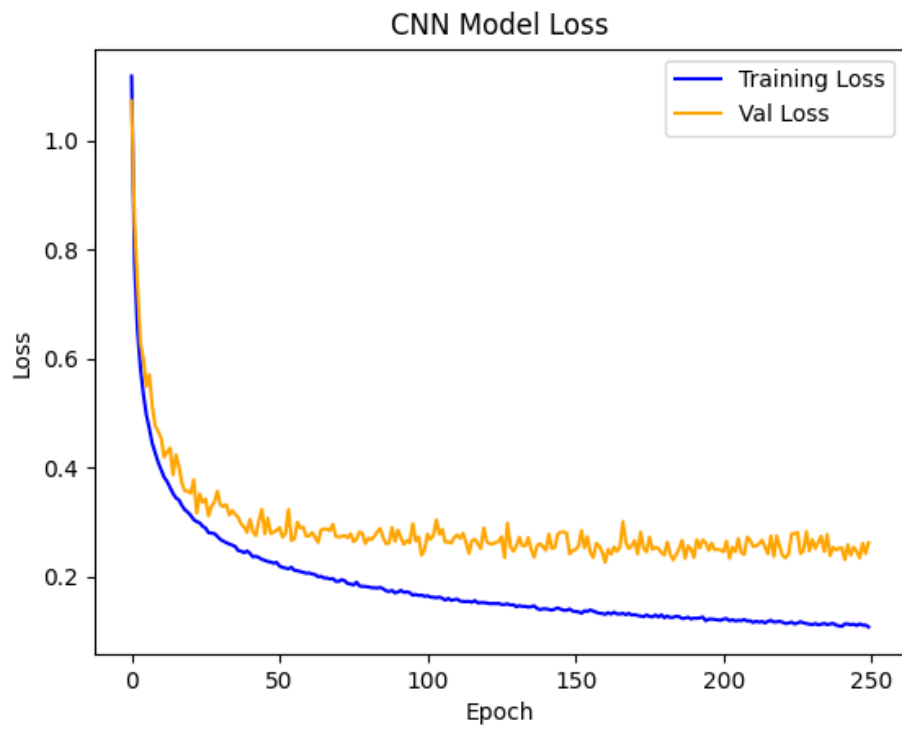


Figure 11 Gesture Prediction Model Loss

When we tested the model on data not included in the dataset and taken on a different day with varying EMG armband placement, the resulting confusion matrix seen in Figure 12 was calculated. Based on this test data, our model has an overall accuracy of ~93%. Individual gesture accuracy can be found in Table 3.

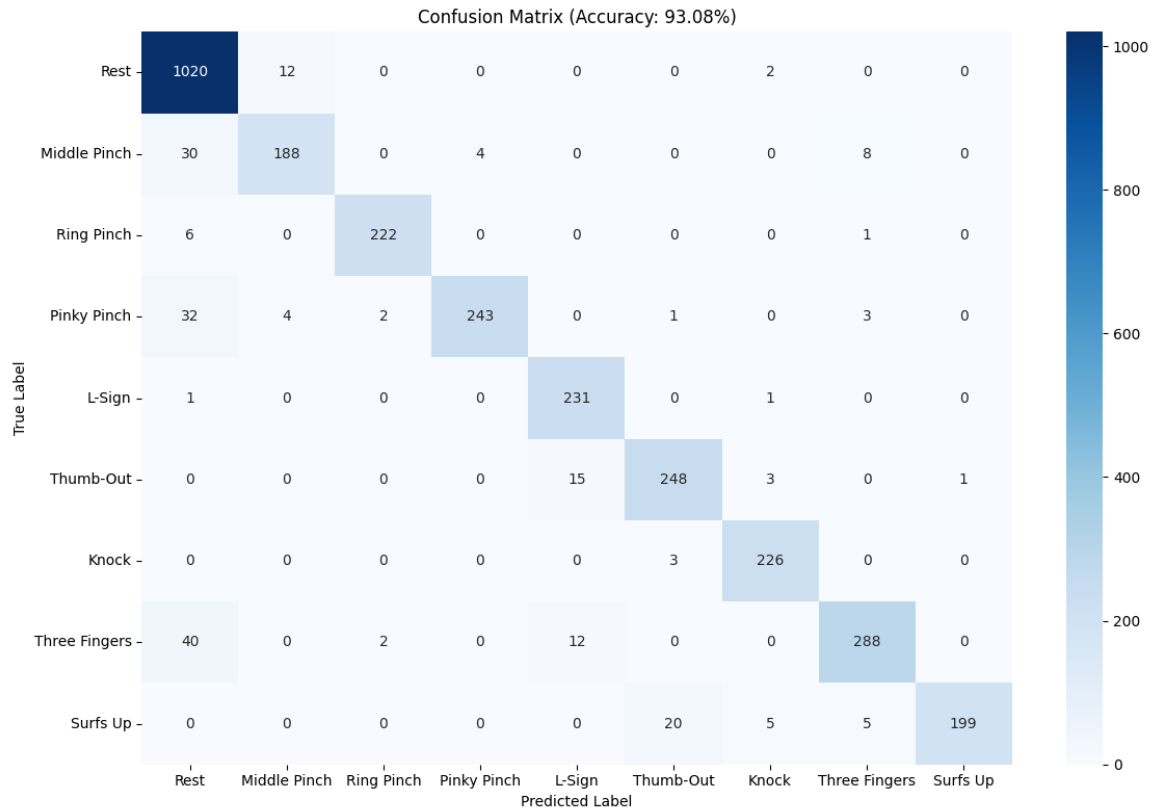
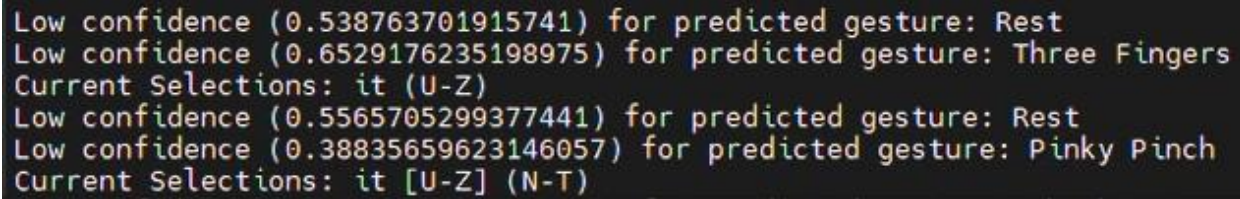


Figure 12 Gesture Prediction Confusion Matrix

User Application Design

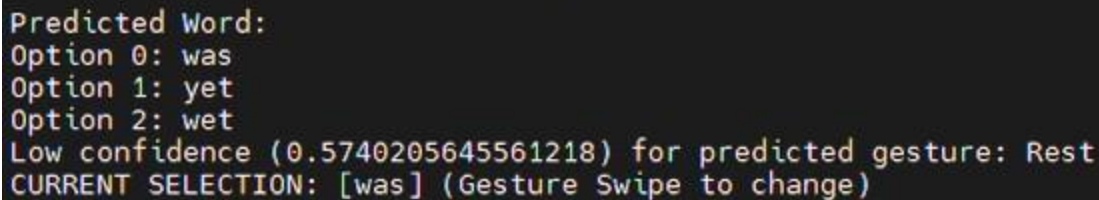
Overview of the User Interface (UI)

Our user interface operates primarily through a terminal-based display on the Raspberry Pi. The interface is highly responsive and provides real-time feedback to the user in a text-based format. As each gesture is detected, the UI displays the predicted gesture label with its raw prediction accuracy score. If a letter-based gesture is detected, it will also show the user the current group to typed letters in addition to any previously typed words. Upon submission of the word via the “Knock” gesture, the top three most-likely words and displayed along with instructions to “Swipe” to change words. An example of the UI output during typing can be seen in Figure 13, and an example of the output after word prediction can be seen in Figure 14. When a word is chosen, it is concatenated into the full sentence, which remains visible during the duration of typing.



```
Low confidence (0.538763701915741) for predicted gesture: Rest
Low confidence (0.6529176235198975) for predicted gesture: Three Fingers
Current Selections: it (U-Z)
Low confidence (0.5565705299377441) for predicted gesture: Rest
Low confidence (0.38835659623146057) for predicted gesture: Pinky Pinch
Current Selections: it [U-Z] (N-T)
```

Figure 13 UI Terminal Output During Typing



```
Predicted Word:
Option 0: was
Option 1: yet
Option 2: wet
Low confidence (0.5740205645561218) for predicted gesture: Rest
CURRENT SELECTION: [was] (Gesture Swipe to change)
```

Figure 14 UI Terminal Output During Word Prediction and Selection

Primary Functions and Controls

The primary functions of the UI are designed to be entirely hands-free, utilizing only user gestures and their mapped commands. The specific functions are as follows:

- **Power/ Session Toggle:** The user may perform the “Surfs Up” gesture to wake the device and initialize typing. The user can perform the gesture command again to reset and terminate the typing program.
- **Letter Input:** Letter groups are selected using the “Middle Pinch”, “Ring Pinch”, “Pinky Pinch”, and “Three Fingers” gesture commands. The order the user does the gestures determines the order the letters appear.
- **Word Selection:** After the predicted words are displayed, the user can perform the “L-Sign” gesture to swipe between the options until their desired word is selected in the terminal.
- **Confirmation:** The “Knock” gesture acts as the spacebar and enter commands. It is used to submit the letter groups to the word prediction model, and to confirm the user’s selected words choice after prediction. After word confirmation, the terminal resumes regular typing.
- **Correction/ Deletion:** The “Thumb-Out” gesture functions as a backspace or delete button. This allows the user to delete any accidental or incorrectly added letter groups. It also allows the user to delete words during the typing process if an incorrect word was predicted or added.
- **State Control:** To ensure smooth transitions between each gesture prediction, the “Rest” gesture position must be detected between each command. Without it, the model may make predictions during transitional periods that lead to inaccurate or unwanted commands.

Technical Implementation Details

The backend of the UI is a Python hardware interfacing script that collects data vis the MindRove API to stream the eight channels of EMG data through the gesture and word prediction process. The real-time application relies on EMA voting configured for 1-second samples to ensure stability and fluidity during user input.

The gesture classification runs locally using a TensorFlow deep learning framework. For word generation, the application functions as a client that interfaces with the FlexType remote word prediction model. A communication module uses the Python *requests* library to package mapped gesture coordinates as a JSON payload and executes POST commands to the REST API hosted at *imagineville.org*. The API then returns the top three predicted words, which the driver pushes to terminal display.

Results, Performance, and Requirements Compliance

Results Summary and Evidence

The final product was able to predict 8 different gestures with an overall accuracy of 93.08%, as seen in Figure 12, using the commercial MindRove EMG Sensor. It was able to map these eight gestures to different commands and letter groups and was able to output the three most likely predicted word based on the gestures performed by the user. However, the gesture prediction model was trained using mostly EMG data from a singular subject, therefore, these results were achieved when they were the user of the device and had lower accuracy when other people used the device. Regardless, it was still able to predict gestures and output the desired word of the user. As far as the PCB hardware is concerned, the device was able to successfully collect EMG data from users but was unable to process it correctly due to a missing offset in the circuit.

Product Requirement Compliance

The following list details the requirements for our designed device:

- **The EMG wristband shall convert the analog EMG signal into a digital signal.**
 - This requirement was met since the PCB contains an ADC that converts the analog EMG signals into digital signals.
- **The EMG and IMU sensors will share data with the user's device.**
 - This requirement was partially met. The EMG sensor gathers EMG data and sends it to the Raspberry Pi for processing. However, we didn't need IMU data for the gesture prediction model, so we did not collect or send any IMU data
- **The gesture classification algorithm shall have an accuracy of at least 90%.**
 - This requirement was met. The gesture classification algorithm achieved an accuracy of 93.08%, which can be seen in Figure 12.
- **The word prediction algorithm shall have an accuracy of at least 90%.**
 - This requirement was met. The FlexType word prediction model has over a 90% accuracy, which was confirmed by its creator, Dr. Dylan Gaines.
- **The wearable device shall read gestures when a user's arm is in predetermined position.**
 - This requirement was mostly met. The gesture recognition could identify eight different gestures in a predetermined arm position. However, this was not the case for the "Rest" gesture as it was only detected when having the arm down instead of up.
- **The gesture model shall wait for the gesture that indicates a space or period to begin sending the saved data to the word prediction model.**

- This requirement was met. The “Knock” gesture is used for the “Space/ Enter” command, which corresponds to sending the information to the word prediction model and getting the three most likely words.

```
1/1 ————— 1s 529ms/step
Low confidence (0.5116177201271057) for predicted gesture: Knock
Run = True, Wait = False, Typing = True, Checking = False

Predicted Word:
Option 0: hello
Option 1: mails
Option 2: legit
```

Figure 15 Knock Gesture Output

- The model shall include a “Backspace” gesture for incorrectly typed words and incorrectly predicted gestures.
 - This requirement was met. The “Thumb-Out” gesture is used as a backspace button allowing the user to delete a word or letter that they incorrectly spelled or added.

```
Low confidence (0.38835659623146057) for predicted gesture: Pinky Pinch
Current Selections: it [U-Z] (N-T)
Low confidence (0.48308300971984863) for predicted gesture: Thumb-Out
Low confidence (0.7375234365463257) for predicted gesture: L-Sign
Low confidence (0.5045080184936523) for predicted gesture: Rest
Low confidence (0.46102187037467957) for predicted gesture: Thumb-Out
Current Selections: it (U-Z)
```

Figure 16 Output of Thumb Out Gesture Deleting Most Recent Letter Group

- The model shall include a “Swipe” gesture to update incorrectly predicted words with the next best prediction.
 - This requirement was met. The “L-Sign” gesture is used to switch between the three most likely words that the word prediction model outputs.


```
Predicted Word:
Option 0: is
Option 1: it
Option 2: in
Low confidence (0.4660458564758301) for predicted gesture: Rest
CURRENT SELECTION: [is] (Gesture Swipe to change)
CURRENT SELECTION: [is] (Gesture Swipe to change)
CURRENT SELECTION: [is] (Gesture Swipe to change)
CURRENT SELECTION: [is] (Gesture Swipe to change)
CURRENT SELECTION: [is] (Gesture Swipe to change)
CURRENT SELECTION: [is] (Gesture Swipe to change)
CURRENT SELECTION: [is] (Gesture Swipe to change)
CURRENT SELECTION: [is] (Gesture Swipe to change)
Low confidence (0.7351486682891846) for predicted gesture: L-Sign
CURRENT SELECTION: [is] (Gesture Swipe to change)
Swiped! New Selected Word: [it]
```

Figure 17 Output of L-Sign Swiping to the Next Word

- **The EMG sensor shall read a user's gesture within a 1 second period.**
 - This requirement was met. The system takes around 0-1s to predict a gesture

```
1/1 ————— 1s 832ms/step
Run = True, Wait = True, Typing = False, Checking = True
1/1 ————— 1s 601ms/step
Run = True, Wait = True, Typing = False, Checking = True
1/1 ————— 0s 338ms/step
Run = True, Wait = True, Typing = False, Checking = True
1/1 ————— 0s 158ms/step
Run = True, Wait = True, Typing = False, Checking = True
1/1 ————— 0s 165ms/step
Run = True, Wait = True, Typing = False, Checking = True
1/1 ————— 0s 200ms/step
Run = True, Wait = True, Typing = False, Checking = True
1/1 ————— 0s 195ms/step
Low confidence (0.5386517643928528) for predicted gesture: Rest
```

Figure 18 Gesture Prediction Output with Time Information

- **The gesture classification and word prediction models may run locally on the user's device.**
 - This requirement was not met. Due to missing files from the word prediction model, the model was unable to run locally and had to be accessed remotely.

Improvements and Optimizations

If given more time, some improvements or optimizations that could enhance the system's performance are:

- Optimizing what type of gesture is being performed. Some gestures were too similar which made it hard for the model to differentiate between them. Also, some gesture like "Wiggle Fingers" that we used in training were more movements rather than a gesture being held which confused the model.
- Obtain more data from different individuals in order to generalize the model to work on anyone with high accuracy.
- Obtain data from individuals with neurological disorders, since their group is the motivation for our project, we would like the model to work on individuals with difficulties with fine motor control.
- Reduce the number of channels needed for gesture detection to make the device smaller and easier to wear.

Conclusions and Lesson Learned

Throughout the design process of our product, we learned many practical and technical skills associated with technological innovation. We learned how to build a custom dataset, how to build and improve AI model performance, and how to design and build custom hardware. We also learned how to perform market research, plan based on project requirements, and how to pivot when unforeseeable challenges arise.

Overall, our end product had many successes. We achieved a highly accurate gesture prediction model based on a custom built dataset. We also achieved a fully functional device that could convert each gesture to its command function and return predicted words, with quick predictions being made and updated every second. While we were not able to fully implement our designed hardware device, our electrode PCBs were able to get a mostly accurate signal and could connect to the control board PCB.

Our biggest challenges were with building our dataset and putting our hardware together. For our dataset, we initially planned on having a larger pool of volunteers, aiming to have between 30-40 potential subjects. However, many people did not have time or were not interested in allowing us to take our data. To adjust for this, we instead changed to primarily taking data from one unique subject and building a larger database tuned to them. This allowed us to continue improving our model without relying on outside volunteers. We also added data augmentation to give the illusion of more varied data, which helped to boost training accuracy.

Our hardware issues were largely due to issues out of our control, such as shipping speeds. We knew that it may take some time to get our components, so we ordered everything as early as possible. Due to shipping errors with the company, we ended up getting most of our components almost a month late and even had to reorder some of them to try to get what could as early as possible. Because of our decreased time with the full components and because none of us had ever soldered before, it took much longer than anticipated to have our hardware put together. Though we were not able to

implement our hardware with our software and ended up using a commercial EMG device, we were able to prove that our designed hardware could pick up a signal. The signal was accurate, but due to missing an offset component, could only produce square waves instead of sine waves.

If given more time for this project, we would have liked to implement a cleaner and prettier UI to make the project look more professional and user friendly. We also would have liked to improve the timing of the gesture prediction during the typing stage to decrease the time between each user input. We also would have liked to fully integrate our hardware with our software, though we are proud to know that our hardware was mostly functional, excluding the missing offset. Overall, we are pleased with our final product and are happy to have met most of our requirements.

Appendix

Appendix A

Link to GitHub Repository

<https://github.com/JuliaCJ/FlexType-EMG-Keyboard/tree/main/Gestures>

Appendix B

Model Architecture Code Snippet

```
137  def build_model():
138      model_cnn = Sequential()
139      model_cnn.add(Conv1D(64, kernel_size=15, activation='relu', padding='same', input_shape=(
140          window_size, len(filtered_cols)))) # (Sampling rate = 500Hz * Window length = 250ms, 8 channels)
141      model_cnn.add(MaxPooling1D(2))
142
143      model_cnn.add(Conv1D(64, kernel_size=9, activation='relu', padding='same'))
144      model_cnn.add(BatchNormalization())
145      model_cnn.add(MaxPooling1D(2))
146      model_cnn.add(Dropout(0.4))
147
148      model_cnn.add(Conv1D(64, kernel_size=5, activation='relu', padding='same'))
149      model_cnn.add(BatchNormalization())
150      model_cnn.add(MaxPooling1D(2))
151      model_cnn.add(Dropout(0.4))
152
153      model_cnn.add(GlobalAveragePooling1D())
154      model_cnn.add(Dense(64, activation='relu'))
155      model_cnn.add(Dense(num_gestures, activation='softmax'))
156
157      model_cnn.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
158      model_cnn.summary()
159      return model_cnn
160
```

Appendix C

EMA Voting Algorithm Code Snippet

```

134     EMA_DECAY = 0.25          # How much weight new predictions carry vs history
135     PROB_TEMP = 0.8
136     ENTER_THRESHOLD = 0.5     # Confidence needed to switch to a new gesture
137     EXIT_THRESHOLD = 0.2     # Confidence drop needed to revert to Rest
138     NUM_CLASSES = len(gesture_strings)
139
140     # Decalre Voter Class
141     class GestureVoter:
142     def __init__(self, num_classes, decay=0.2, thresholds=None):
143         self.ema_probs = np.zeros(num_classes)
144         self.decay = decay
145         self.current_stable = 0
146         self.initialized = False
147         self.default_threshold = 0.45
148         self.thresholds = thresholds if thresholds else {}
149
150     def predict_voted_gesture(self, new_probs, temperature=0.8):
151         sharpened_probs = np.power(new_probs, 1 / temperature)
152         sharpened_probs /= np.sum(sharpened_probs)
153         if not self.initialized:
154             self.ema_probs = sharpened_probs
155             self.initialized = True
156         else:
157             self.ema_probs = (1 - self.decay) * self.ema_probs + self.decay * sharpened_probs
158
159         best_gesture = np.argmax(self.ema_probs)
160         best_confidence = self.ema_probs[best_gesture]
161         thresh = self.thresholds.get(best_gesture, self.default_threshold)
162
163         if best_gesture != self.current_stable:
164             if best_confidence >= thresh:
165                 self.current_stable = best_gesture
166         else:
167             if self.current_stable != 0:
168                 if self.ema_probs[0] > 0.6:
169                     self.current_stable = 0
170         return self.current_stable

```